School of Computer Science

FACULTY OF ENGINEERING AND PHYSICAL SCIENCES

UNIVERSITY OF LEEDS

Technical Report

Premier League Analytics API

Rakan Rouchdi

COMP3011 – Web Services and Web Data

Submission: Individual Coursework API Project

Table of Contents

1. Introduction iii

1.1 Project Resources iii

2. System Architecture iv

3. Technology Stack iv

4. Database Design v

5. Dataset Integration v

6. API Design vi

7. Testing Approach vi

8. Challenges and Lessons vii

9. Limitations and Future Work vii

10. Generative AI Declaration vii

References viii

Appendix ix

Appendix A – API Documentation ix

Appendix B – Example API Request and JSON Response ix

Appendix C – Frontend Dashboard Screenshots xi

Appendix D – Github Commit History Overview xiv

Appendix E – Generative AI Usage xv

1. Introduction

This project presents the design and implementation of a RESTful web API for analysing English Premier League football data. The system provides structured access to information about players, teams, and matches while also offering analytical insights such as top goal scorers, league tables, and team performance summaries.

The primary objective of the project was to design and implement a modular backend system that demonstrates key web service development practices including database integration, API design, automated testing, and documentation. The API was implemented using the FastAPI framework and integrates publicly available football datasets obtained from Kaggle.

To demonstrate how the API can be used in a real application, a lightweight browser-based dashboard was also developed using HTML, CSS, and JavaScript. The dashboard consumes the API endpoints and allows users to explore players, teams, matches, and analytics through a graphical interface.

The API has also been deployed to the cloud using the Render hosting platform. This deployment demonstrates how the system can operate in a publicly accessible environment and allows the API to be accessed through a live URL.

1.1 Project Resources

The following resources provide access to the implementation and supporting materials for this project.

Github Repository

The complete source code for the project, including backend implementation, automated tests, and documentation, is available at: https://github.com/Rakan-Rouchdi/premier-league-api

The repository contains the full commit history demonstrating version control practices during development.

API Documentation

Interactive API documentation is automatically generated using FastAPI's OpenAPI integration. The documentation allows developers to explore all available endpoints, parameters, and example responses.

The documentation can be accessed at: https://premier-league-api-re2e.onrender.com/docs

A PDF version of the API documentation is also included in the repository:

https://github.com/Rakan-Rouchdi/premier-league-api/blob/main/docs/API_Documentation.pdf

Live API Deployment

The deployed API can be accessed at: https://premier-league-api-re2e.onrender.com/

Presentation Slides

Presentation slides used for the oral demonstration of the project are available at:

Premier League API.pptx

2. System Architecture

The application follows a modular architecture designed to ensure separation of concerns and maintainability. The system is organised into several logical layers that manage different responsibilities within the application.

The **router layer** defines the public API endpoints and handles incoming HTTP requests. Each domain of the system, including players, teams, matches, and analytics, is implemented within a separate router module. This organisation improves code readability and allows the API to be extended more easily.

The **CRUD layer** contains reusable database operations responsible for retrieving, creating, updating, and deleting records. Separating CRUD operations from routing logic allows database interactions to remain modular and easier to maintain.

The **database layer** manages the connection to the SQLite database and defines SQLAlchemy models representing the core entities of the system. These models map Python classes to relational database tables and enable structured queries.

The **schema layer** uses Pydantic models to validate request data and define the structure of API responses. This ensures that requests follow the expected format and that responses remain consistent.

The **services layer** manages background tasks such as importing external datasets into the database.

This layered architecture ensures that API routing, validation, business logic, and data storage remain clearly separated.

3. Technology Stack

Several modern technologies were used to implement the API.

FastAPI was selected as the backend framework due to its high performance, automatic OpenAPI documentation generation, and strong integration with Python typing and Pydantic validation models. FastAPI also provides an intuitive development workflow that simplifies the creation of RESTful APIs.

SQLite was chosen as the relational database for this project. SQLite is lightweight and requires minimal configuration, making it suitable for coursework projects and small-scale applications.

SQLAlchemy was used as the Object Relational Mapping (ORM) library. It allows database operations to be performed using Python objects instead of raw SQL queries. This improves code readability and maintainability while still allowing complex queries to be executed efficiently.

Pydantic was used to define request and response schemas. It performs automatic data validation and ensures that API responses follow a consistent structure.

Pytest was used to implement automated testing for the API. Automated tests verify that endpoints behave correctly and help detect issues during development.

FastAPI automatically generates interactive API documentation through **Swagger UI** and **ReDoc**, allowing developers to explore and test the API through a browser interface.

4. Database Design

The database schema consists of three primary entities: **Teams**, **Players**, and **Matches**.

The **Teams table** stores information about football clubs participating in the Premier League. Each team is assigned a unique identifier and name.

The **Players table** stores player statistics including appearances, minutes played, goals, assists, and playing position. Each player record is associated with a team through a foreign key relationship linking the Players table to the Teams table.

The **Matches table** stores match-level statistics including home and away teams, goals scored, shots, fouls, corners, and disciplinary records. Each match references two teams through foreign keys representing the home and away teams.

This relational design enables efficient querying and supports analytical operations such as calculating league tables, identifying top scorers, and analysing match outcomes.

5. Dataset Integration

Two publicly available datasets from Kaggle were used to populate the database.

The first dataset contains historical Premier League match data covering seasons from 2000 to 2025. This dataset includes match results and match-level statistics such as goals scored, shots, fouls, and disciplinary actions.

The second dataset contains player statistics for the 2024/25 season including appearances, minutes played, goals, assists, and playing positions.

A custom import script was implemented to load these CSV datasets into the database. During the import process, unique teams were extracted and inserted into the Teams table before importing related player and match records. This ensured that foreign key relationships between tables were correctly maintained.

Generative AI tools were used during dataset exploration to analyse the dataset structure and assist in generating the initial import scripts used to populate the database.

6. API Design

The API follows RESTful design principles and exposes endpoints grouped by resource type.

The **Players resource** provides full CRUD functionality. Users can create, retrieve, update, and delete player records through standard HTTP methods. This satisfies the requirement of implementing a data model with full database-backed CRUD functionality.

The **Teams resource** allows retrieval of team information and associated data.

The **Matches resource** provides access to match statistics and supports filtering by season or team identifier.

In addition to basic CRUD functionality, the API includes several **analytical endpoints**. These endpoints compute insights derived from stored match and player data. Examples include retrieving top goal scorers, top assist providers, highest scoring matches, league tables for a specific season, and team performance summaries.

All endpoints return structured JSON responses and follow standard HTTP status code conventions such as 200 for successful requests and 201 for newly created resources.

7. Testing Approach

Automated testing was implemented using Pytest to verify that the API behaves correctly.

Tests were written to validate key functionalities including player CRUD operations, team retrieval endpoints, match retrieval endpoints, and analytics endpoints.

The root API endpoint was also tested to ensure that the API starts correctly.

Automated testing improves reliability and ensures that future modifications do not introduce unintended behaviour.

8. Challenges and Lessons

Several challenges were encountered during development.

One challenge involved ensuring compatibility between SQLAlchemy database models and Pydantic response schemas. ORM compatibility needed to be enabled so that database objects could be converted into structured API responses.

Another challenge involved importing external datasets while maintaining consistent relationships between teams, players, and matches. Dataset inconsistencies such as variations between team names (for example “Man City” and “Manchester City”) required preprocessing to ensure consistent database entries.

Developing analytical endpoints also required implementing aggregation queries to compute league tables and team statistics.

Through this project, valuable experience was gained in API design, relational database modelling, and automated testing.

9. Limitations and Future Work

Although the API provides useful functionality, several improvements could be implemented in future versions.

Authentication and authorization mechanisms could be introduced to restrict write operations and protect sensitive endpoints.

Additional analytics could be developed, including advanced player statistics, predictive match analysis, and team comparison tools.

Caching mechanisms could also be implemented to improve performance for computationally expensive analytical queries.

10. Generative AI Declaration

Generative AI tools were used during several stages of this project to assist with planning, development, and documentation.

Tools used included ChatGPT for planning, research and refining drafts. Cursor was also used for coding assistance.

AI assistance was used during the project planning stage to explore possible technology stacks and architectural approaches for building a RESTful web API. This helped evaluate frameworks, database options, and overall system structure before implementation began.

Generative AI tools were also used during dataset discovery and preparation. AI assistance helped identify relevant publicly available datasets related to English Premier League statistics. AI was additionally used to analyse dataset structure and generate initial scripts for importing CSV data into the SQL database.

During development, a generative AI coding assistant was used to support the implementation of the front-end dashboard. AI-generated suggestions were used to help structure parts of the HTML, CSS, and JavaScript code required to build the browser-based interface that interacts with the API.

Generative AI was also used during the documentation process to assist with drafting sections of this technical report and refining grammar and clarity of written explanations.

All architectural decisions, integration of generated code, debugging, testing, and final implementation were performed independently by the author. AI outputs were reviewed, modified, and integrated where appropriate and were used as development assistance rather than direct replacements for independent work. The use of generative AI therefore served as a productivity and learning tool.

References

FastAPI Documentation

https://fastapi.tiangolo.com/

SQLAlchemy Documentation

https://docs.sqlalchemy.org/en/20/

Pytest Documentation

https://docs.pytest.org/en/stable/

Kaggle: English Premier League Match Data (2000–2025)

https://www.kaggle.com/datasets/marcohuiii/english-premier-league-epl-match-data-2000-2025

Kaggle: English Premier League Player Statistics (2024/25)

https://www.kaggle.com/datasets/aesika/english-premier-league-player-stats-2425

Appendix

Appendix A – API Documentation

The API documentation for this project is automatically generated using FastAPI's OpenAPI integration. This documentation provides a structured overview of all available endpoints, request parameters, and response schemas.

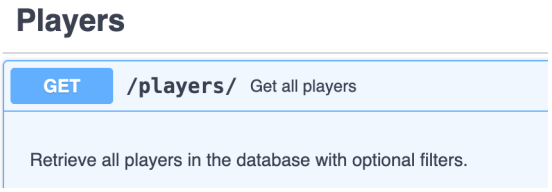
Interactive documentation can be accessed through the deployed API: https://premier-league-api-re2e.onrender.com/docs

A static version of the documentation is included in the repository: docs/API_Documentation.pdf

The documentation allows developers to explore and test endpoints directly through the browser interface.

Appendix B – Example API Request and JSON Response

Example 1 – Retrieve Players



Example Response:

Code	Details
200	Response body


```
[
  {
    "name": "Ben White",
    "team_id": 1,
    "position": "DEF",
    "appearances": 17,
    "minutes": 1198,
    "goals": 0,
    "assists": 2,
    "id": 1
  },
  {
    "name": "Bukayo Saka",
    "team_id": 1,
    "position": "MID",
    "appearances": 25,
    "minutes": 1735,
    "goals": 6,
    "assists": 10,
    "id": 2
  },
```


Example 2 – Retrieve League Table

GET `/analytics/league-table` Season league table

Generate a league table based on match results for a specific season.

Parameters

Name	Description
season * required `string` *(query)*	Season format example: 2024/25 2024/25

Example Response:

Code	Details
200	Response body


```
[
  {
    "team_id": 25,
    "team_name": "Liverpool",
    "played": 35,
    "wins": 25,
    "draws": 7,
    "losses": 3,
    "goals_for": 81,
    "goals_against": 35,
    "goal_difference": 46,
    "points": 82
  },
  {
    "team_id": 1,
    "team_name": "Arsenal",
    "played": 35,
    "wins": 18,
    "draws": 13,
    "losses": 4,
    "goals_for": 64,
    "goals_against": 31,
    "goal_difference": 33,
    "points": 67
  }
```

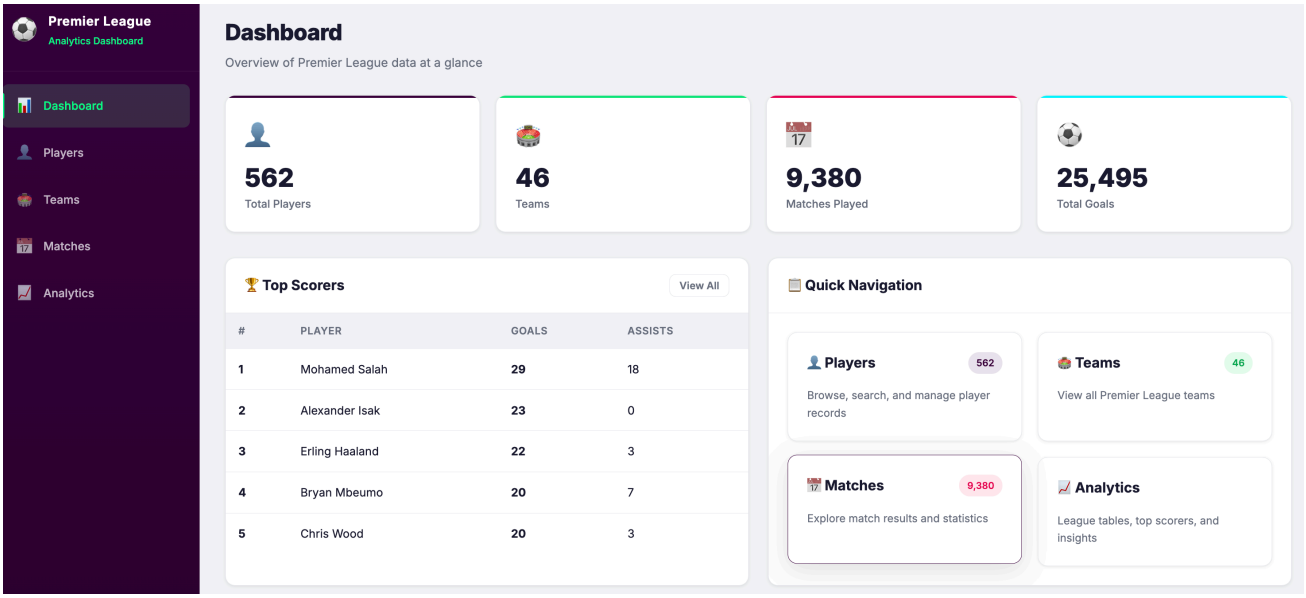

Appendix C – Frontend Dashboard Screenshots

A lightweight browser-based dashboard was developed to demonstrate how the API can be consumed by a client application.

The dashboard was implemented using HTML, CSS, and JavaScript and communicates with the backend API through HTTP requests.

The interface allows users to:

- browse players
- view team information
- explore match statistics
- view analytical insights such as top scorers and league tables


Premier League
Analytics Dashboard
Dashboard
Players
Teams
Matches
Analytics
Dashboard
Overview of Premier League data at a glance
562
Total Players
46
Teams
9,380
Matches Played
25,495
Total Goals
Top Scorers
View All
PLAYER GOALS ASSISTS
1 Mohamed Salah 29 18
2 Alexander Isak 23 0
3 Erling Haaland 22 3
4 Bryan Mbeumo 20 7
5 Chris Wood 20 3
Quick Navigation
Players 562
Browse, search, and manage player records
Teams 46
View all Premier League teams
Matches 9,380
Explore match results and statistics
Analytics
League tables, top scorers, and insights

Premier League
Analytics Dashboard

Dashboard
Players
Teams
Matches
Analytics

Players

Browse, search, and manage player records

Search players by name... | All Teams | All Positions | + Add Player

ID	NAME	TEAM	POSITION	APPS	MINS	GOALS	ASSISTS	ACTIONS
1	**Ben White**	Arsenal	DEF	17	1,198	**0**	2	Edit Delete
2	**Bukayo Saka**	Arsenal	MID	25	1,735	**6**	10	Edit Delete
3	**David Raya**	Arsenal	GKP	38	3,420	**0**	0	Edit Delete
4	**Declan Rice**	Arsenal	MID	35	2,833	**4**	7	Edit Delete
5	**Ethan Nwaneri**	Arsenal	MID	26	889	**4**	0	Edit Delete
6	**Gabriel Jesus**	Arsenal	FWD	17	603	**3**	0	Edit Delete
7	**Gabriel Magalhães**	Arsenal	DEF	28	2,365	**3**	1	Edit Delete
8	**Gabriel Martinelli**	Arsenal	MID	33	2,300	**8**	4	Edit Delete
9	**Jakub Kiwior**	Arsenal	DEF	17	1,117	**1**	0	Edit Delete
10	**Jorginho**	Arsenal	MID	15	702	**0**	0	Edit Delete

Premier League
Analytics Dashboard

Dashboard
Players
Teams
Matches
Analytics

Teams

All Premier League teams in the database

Search teams...

Arsenal ID: 1
Click to view team details and performance summary

Aston Villa ID: 2
Click to view team details and performance summary

Birmingham ID: 3
Click to view team details and performance summary

Blackburn ID: 4
Click to view team details and performance summary

Blackpool ID: 5
Click to view team details and performance summary

Bolton ID: 6
Click to view team details and performance summary

Bournemouth ID: 7
Click to view team details and performance summary

Bradford ID: 8
Click to view team details and performance summary

Brentford ID: 9
Click to view team details and performance summary

Brighton ID: 10
Click to view team details and performance summary

Burnley ID: 11
Click to view team details and performance summary

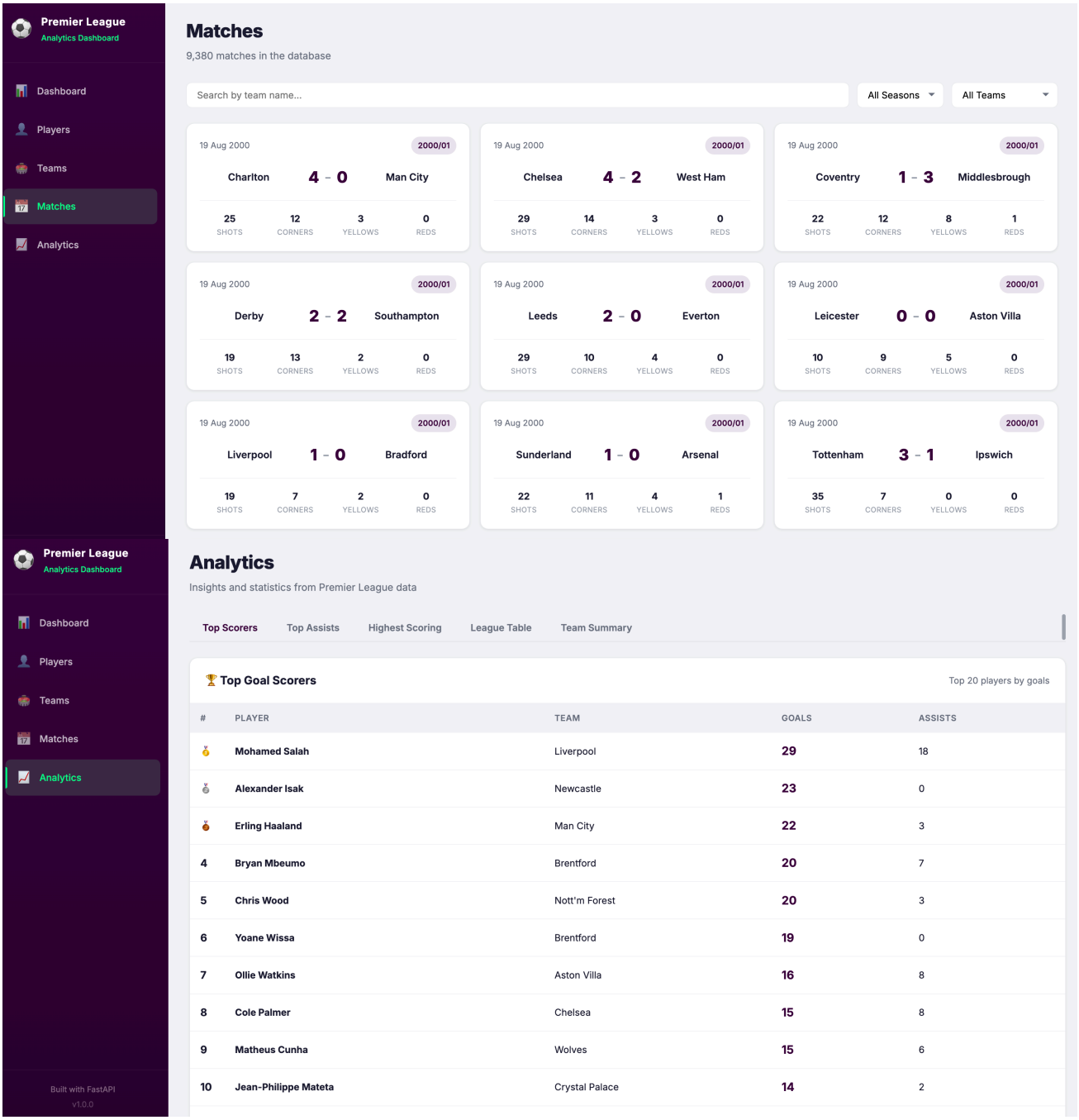
Cardiff ID: 12
Click to view team details and performance summary

Charlton ID: 13
Click to view team details and performance summary

Chelsea ID: 14
Click to view team details and performance summary

Coventry ID: 15
Click to view team details and performance summary

Crystal Palace ID: 16
Click to view team details and performance summary



These screenshots demonstrate how the API can be integrated into a user-facing interface.

Appendix D – Github Commit History Overview

The full implementation of the project is hosted on GitHub.

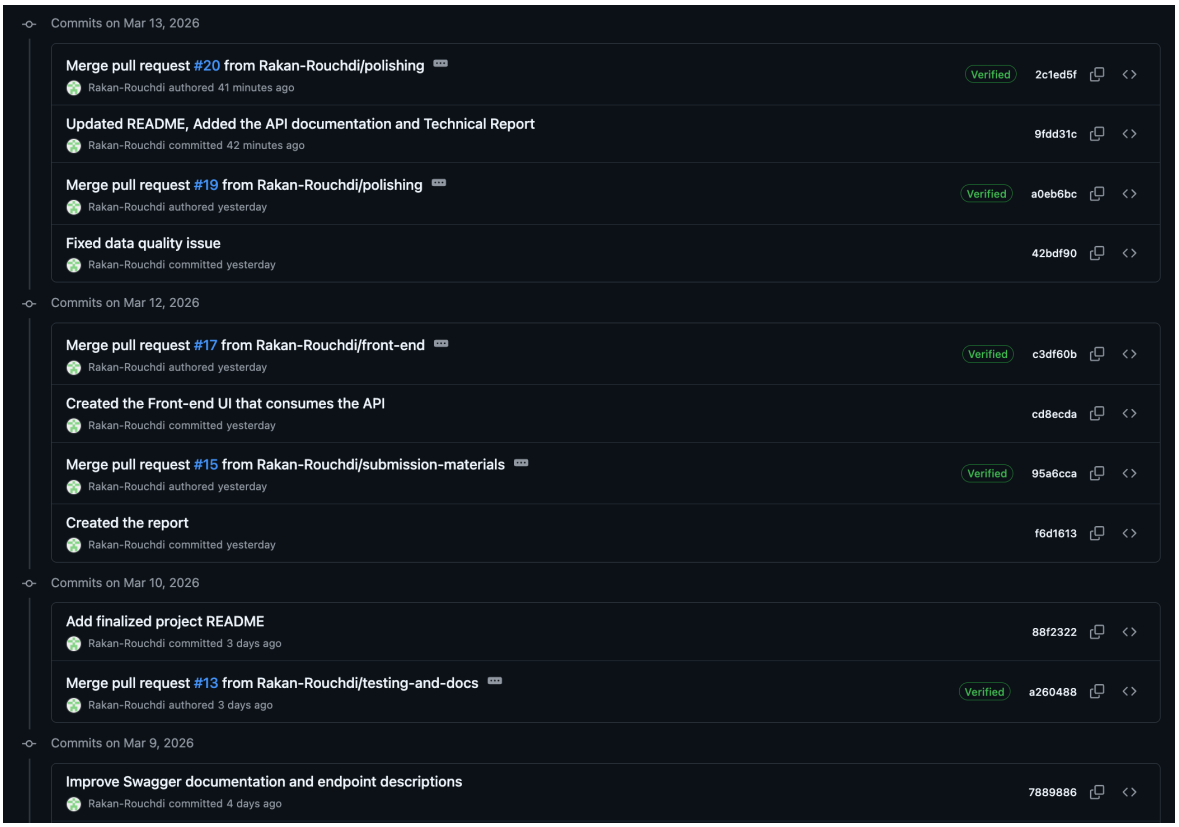
https://github.com/Rakan-Rouchdi/premier-league-api

The repository contains:

- backend API implementation
- database models
- automated tests

- frontend dashboard
- documentation and report materials

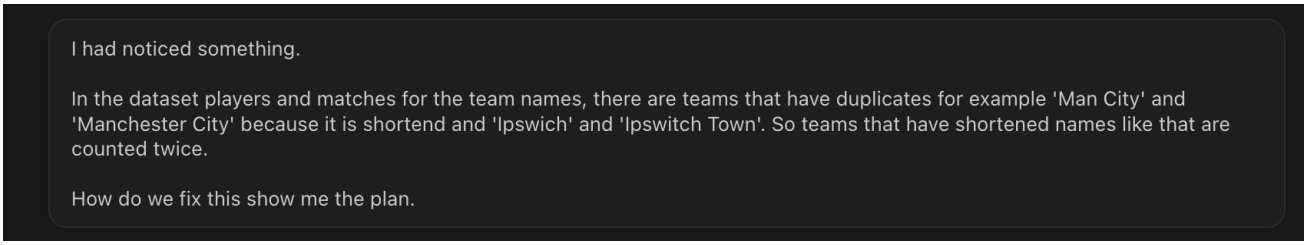
The commit history demonstrates the development process and incremental implementation of the project features.



Appendix E – Generative AI Usage

Generative AI tools were used during several stages of the development process to support planning, dataset preparation, coding assistance, and documentation.

Examples of Conversations:



Plan: Fix Duplicate Team Names

Information Gathered

- **Matches CSV** uses short names (e.g., `Man City`, `Nott'm Forest`, `Wolves`)
- **Players CSV** uses full names (e.g., `Manchester City`, `Nottingham Forest`, `Wolverhampton Wanderers`)
- The current `import_teams()` function treats them as separate teams, creating 56 teams instead of 46
- Players are linked to the full-name team IDs, matches are linked to the short-name team IDs
- This means analytics (team summary, league table) don't combine match data with player data for the same team

Fix Strategy

Add a **team name mapping dictionary** in `app/services/import_data.py` that normalizes all team names to a single canonical name **before** inserting into the database. This way:

- Both CSVs map to the same team record
- Players and matches share the same `team_id`
- Analytics correctly combine all data

Team Name Mapping (Player CSV → Match CSV canonical name)

```
TEAM_NAME_MAP = {
    "Brighton & Hove Albion": "Brighton",
    "Ipswich Town": "Ipswich",
    "Leicester City": "Leicester",
    "Manchester City": "Man City",
    "Manchester United": "Man United",
    "Newcastle United": "Newcastle",
    "Nottingham Forest": "Nott'm Forest",
    "Tottenham Hotspur": "Tottenham",
    "West Ham United": "West Ham",
    "Wolverhampton Wanderers": "Wolves",
}
```

Files to Edit

File	Change
`app/services/import_data.py`	Add `TEAM_NAME_MAP` dict, apply normalization in `import_teams()`, `import_players()`, and `import_matches()`
`premier_league.db`	Delete and re-import (re-run `import_data.py` to rebuild with correct mappings)

Okay we locked in the Premier league datasets and the tech stack. Show me our new full project plan and folder structure so we can tackle this methodologically

Perfect. We'll **lock the project now** and make it **simple, strong, and mark-focused**.

This plan is designed to help you aim for a **top mark**, while still being realistic for a student project and beginner-friendly in execution.

---

1. Final locked project

Project title

Premier League Statistics API

Project idea

A **REST API** built with **FastAPI + SQLite** that uses:

- a **Premier League matches dataset (2000–2025)** for historical match analytics
- a **Premier League player stats dataset (2024/25)** for player data and CRUD

Okay currently, I don't have a front-end (Web-App) for the API. I just have the openAPI, swagger. So now I want to make a front-end design. So make that for me but show me the plan first and how you will do it and with what tech stack.

Front-End Plan: Premier League Analytics Web App

Tech Stack

Component	Technology	Rationale
Markup	HTML5	No build tools needed, simple to deploy
Styling	CSS3 (custom, no framework)	Clean, modern look with Premier League theming
Interactivity	Vanilla JavaScript (ES6+)	Calls your FastAPI endpoints via `fetch()`, no npm/Node required
Serving	FastAPI `StaticFiles`	Served from the same server — no separate frontend server needed

Why this stack? Since this is a university coursework project, using plain HTML/CSS/JS keeps things simple, avoids unnecessary complexity (no React/Vue/Angular build pipeline), and clearly demonstrates that your frontend is consuming your REST API. It also means zero additional dependencies.

Pages / Sections

Page	Description	API Endpoints Used
Dashboard (Home)	Welcome page with quick stats overview (total players, teams, matches) and navigation cards	`GET /`, `GET /players`, `GET /teams`, `GET /matches`
Players	Searchable/filterable table of all players. Includes Create, Edit, Delete modals for full CRUD	`GET/POST /players`, `GET/PUT/DELETE /players/{id}`
Teams	Grid/cards of all teams with click-through to team detail	`GET /teams`, `GET /teams/{id}`
Matches	Filterable table (by season, by team) showing match results	`GET /matches`, `GET /matches/{id}`
Analytics	Dashboard with: Top Scorers table, Top Assists table, Highest Scoring Matches, League Table (season selector), Team Summary	All `/analytics/*` endpoints

---

UI Design

- **Theme**: Premier League inspired — dark purple (`#37003c`) header/sidebar, green (`#00ff87`) accents, white content areas
- **Layout**: Sidebar navigation + main content area (responsive)
- **Components**: Data tables, stat cards, filter dropdowns, modal dialogs for CRUD

File Structure

```
frontend/
├── index.html          # Main SPA shell with sidebar navigation
├── css/
│   └── styles.css      # All styling
├── js/
│   ├── app.js          # Main app logic, routing, navigation
│   ├── dashboard.js    # Dashboard page
│   ├── players.js      # Players page + CRUD modals
│   ├── teams.js        # Teams page
│   ├── matches.js      # Matches page
│   └── analytics.js    # Analytics page
```

Backend Changes

- `app/main.py` — Add `StaticFiles` mount to serve the `frontend/` directory and an HTML route to serve `index.html`
- `app/main.py` — Add CORS middleware (in case needed for development)

Dependent Files to Edit

File	Change
`app/main.py`	Add `StaticFiles` mount + CORS middleware

Follow-up Steps

1. Create all frontend files
2. Update `app/main.py` to serve them
3. Start the server and test every page in the browser
4. Verify all CRUD operations and analytics displays work